Faculty/Department:	Faculty of Computing and Intelligent Systems
Program:	B.Tech (Computer Science and Engineering)
Year:	1st
Semester:	I
Course Name:	Software Development with Python
Course Code:	ID1104
Author(s):	Dr. Pankaj Kumar Jadwal
Reviewer(s):	Dr. Santosh Kumar Verma, Jitendra Mathur, Dinesh Kumar Choudhary
Final Review Date:	13/08/2026
Approved By:	Joint BoS (Internal Members)

Course Description

Course Name	Software Development with Python
Course Code	ID1104
Relevant Job Roles & Entrepreneurship Opportunities (If any)	Junior Python Developer / Software Programmer
Relevant QP	SSC/Q0510 Version: 1.0 NSQF Level: 4 (on the scale of 10)
Relevant SDG (If any)	SDG 9 – Industry, Innovation and Infrastructure
Relevant Standards (If any)	PEP 8 – Style Guide for Python Code (naming, formatting, readability) PEP 257 – Docstring Conventions (documentation standards for functions/modules) ISO/IEC 25010 – Software Product Quality Model (covers functionality, reliability, usability — relevant to GUI mini-projects) IEEE 829 – Standard for Software Test Documentation (relevant to the test-case/debugging outcomes) ISO/IEC/IEEE 12207 – Software Life Cycle Processes (relevant to structured development approach)
Competencies	C1. Develop Python-based computational solutions for defined requirements by selecting suitable data representations, program logic, and input-output operations. C2. Implement structured program logic for validation, classification, calculation, and repeated processing using conditional and iterative constructs. C3. Construct data-processing solutions by organising, transforming, and retrieving textual and structured data using appropriate Python collection types. C4. Develop reusable Python application components using functions, functional expressions, modules, and object-oriented constructs for defined software requirements. C5. Create and refine basic desktop applications by integrating graphical interfaces, external data access, introductory concurrency, testing, debugging, and AI-assisted development practices.
Related World Skill Competition	IT Software Solutions for Business
Course Outcomes	After completion of the course, the student will be able to:

	CO1: Develop Python programs for structured computational problems using variables, data types, expressions, type conversion, and input-output operations. CO2: Construct Python solutions using conditional statements, iterative constructs, and loop-control statements for defined logical and repetitive-processing tasks. CO3: Develop Python programs that manipulate strings, lists, tuples, sets, and dictionaries to store, retrieve, transform, and process application data. CO4: Design modular Python programs using user-defined functions, recursion, lambda expressions, built-in functions, modules, classes, and objects. CO5: Develop, test, debug, and refine Tkinter-based desktop applications integrating event handling, external APIs, introductory concurrency, and validated AI-assisted code.

Competency–CO Map Matrix

CO/ Competency	C1	C2	C3	C4	C5
CO1	3	1	0	0	0
CO2	1	3	1	0	0
CO3	1	1	3	1	0
CO4	0	1	1	3	1
CO5	0	1	1	2	3

Mapping scale: 3 = Strong; 2 = Moderate; 1 = Weak; 0 = No relationship.

Course Structure

Total Contact Hrs. (LTP)	Days	Total L&T Hrs.	Total P Hrs.	Total SL Hrs.	Credits
90		30	60	90	4

Assessment Scheme				
Component	**Sub-Component**	**No. of Exams**	**Marks**	**Weigh tage**
Continuous Assessment (CA)	Assignments	1	10	10
	Practical Work	1	20	20
	Viva	1	10	10
	Report Writing	1	10	10
Terminal Assessment (TA)	Practical Exam	1	20	20
	Written Exam	1	30	30
	Total	**6**		**100**

References

1. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
2. Matthes, E. (2023). Python Crash Course: A hands-on, project-based introduction to programming (3rd ed.). No Starch Press.
3. Ramalho, L. (2022). Fluent Python (2nd ed.). O'Reilly Media.
4. Sedgewick, R., Wayne, K., & Dondero, R. (2025). Introduction to Programming in Python: An Interdisciplinary Approach. Pearson.
5. Lutz, M. (2010). Programming Python(4th ed.). O'Reilly Media.

Detailed Syllabus

Unit	Name	Content	Hrs.
I	Python Environment, Data Representation, and Basic Program Development	K1.1: Python programming environment, interpreter, interactive shell, script execution, VS Code, Google Colab, and Python Tutor. K1.2: Program structure, print() function, comments, indentation, syntax errors, and runtime errors. K1.3: Variables, keywords, identifiers, literals, dynamic typing, and dynamic binding. K1.4: Numeric, Boolean, string, and None data types; operators and expressions. K1.5: Input processing, type conversion, formatted output, and simple computational logic. K1.6: AI-assisted code-generation workflow: requirement prompting, code review, execution, validation, and correction of generated code.	5 hrs
		A1.1: Configure Python programming environments in VS Code and Google Colab; execute and trace basic programs using Python Tutor.	8 hrs

		A1.2: Develop input–process–output programs for arithmetic calculations, unit conversion, and student-data processing. A1.3: Trace dynamic typing and dynamic binding through variable reassignment experiments and explain resulting type-related errors. A1.4: Diagnose and correct syntax, type-conversion, and output-formatting errors in supplied Python scripts. A1.5: Generate a simple Python program through an AI coding assistant, test it with varied inputs, identify defects, and refine the final program.	
II	Decision Making, Iteration, and Logical Problem Solving	K2.1: Arithmetic, relational, logical, assignment, membership, and identity operators. K2.2: Boolean expressions, operator precedence, and expression evaluation. K2.3: Conditional constructs: if, if-else, if-elif-else, nested conditions, and structural pattern matching where appropriate. K2.4: Iterative constructs: while, for, nested loops, range(), and sequence traversal. K2.5: Loop-control statements: break, continue, and pass; sentinel-controlled and counter-controlled iteration. K2.6: Algorithm tracing, dry runs, test-case design, and debugging of logical errors in control-flow programs.	6 hrs
		A2.1: Develop programs for classification, validation, grading, billing, and eligibility decisions using conditional statements. A2.2: Construct iterative solutions for numerical series, multiplication tables, pattern generation, and repeated data processing. A2.3: Create a number-guessing application using random-number generation, conditional logic, loops, counters, and loop-control statements. A2.4: Compare alternative solutions using for and while loops for the same computational requirement. A2.5: Debug faulty control-flow programs by tracing execution paths and validating outputs against designed test cases. A2.6: Use an AI coding assistant to propose alternative control-flow solutions, then evaluate their correctness, readability, and boundary-case behaviour.	13 hrs
III	Strings, Collections, Modules, and Data Processing	K3.1: String creation, indexing, slicing, concatenation, formatting, comparison, and built-in string methods. K3.2: Lists: creation, indexing, slicing, traversal, updating, searching, sorting, and list methods. K3.3: Tuples, sets, and dictionaries; characteristics, operations, and suitable application contexts. K3.4: Mutability, immutability, aliasing, variable references, shallow copying, and data-handling implications. K3.5: CRUD operations on lists and dictionaries for structured application data. K3.6: Built-in functions and introductory modules, including math, random, and datetime; importing and using modules.	6 hrs

		K3.7: Selection of suitable collection types for textual, tabular, unique, and key-value data.	
		A3.1: Develop string-processing programs for text cleaning, validation, formatting, searching, and frequency analysis. A3.2: Construct list-based applications for managing to-do items, marks, expenses, or inventory records. A3.3: Develop dictionary-based applications for storing, retrieving, updating, and deleting student or product records. A3.4: Analyse programming requirements and select lists, tuples, sets, or dictionaries with technical justification. A3.5: Demonstrate mutability and variable-reference behaviour through controlled program experiments. A3.6: Integrate built-in modules into small applications such as a password generator, date-based reminder, or random quiz generator. A3.7: Review AI-generated collection-handling code for incorrect data-structure selection, duplicate handling, and missing-key errors.	13 hrs
IV	Functions, Functional Expressions, and Object-Oriented Programming	K4.1: User-defined functions, parameters, arguments, return values, local and global scope, and modular decomposition. K4.2: Recursion, recursive problem formulation, base conditions, recursive tracing, and introductory memoization. K4.3: Lambda expressions and functional-processing tools: map(), filter(), and reduce(). K4.4: Object-oriented programming concepts: classes, objects, attributes, methods, constructors, and instance behaviour. K4.5: Encapsulation, inheritance, method overriding, and polymorphism in simple Python applications. K4.6: Iterators and generators for controlled sequence processing. K4.7: Modular program design through reusable functions, classes, and imported modules.	7 hrs
		A4.1: Design reusable functions for computational, validation, and data-processing requirements using appropriate parameters and return values. A4.2: Develop recursive programs for factorial, Fibonacci sequence, greatest common divisor, and similar structured problems. A4.3: Process collections using lambda expressions with map(), filter(), and reduce(). A4.4: Construct class-based programs to model real-world entities such as bank accounts, library books, student records, or product inventories. A4.5: Extend class-based applications using inheritance and method overriding for related entity types. A4.6: Develop iterator or generator-based solutions for controlled generation and processing of sequence data. A4.7: Refactor a monolithic Python program into modular functions and classes, supported by AI-assisted suggestions that are independently tested and revised.	14 hrs

V	GUI Applications, APIs, Concurrency, Testing, Debugging, and AI-Assisted Development	K5.1: Tkinter fundamentals: application window, widgets, layout managers, event handling, input controls, labels, buttons, and message boxes. K5.2: GUI application design using classes and object-oriented event-driven programming. K5.3: External APIs, HTTP requests, JSON data, API responses, and integration of external data into Python applications. K5.4: Fundamentals of threading and multiprocessing; responsiveness, background tasks, and basic concurrency constraints. K5.5: Test cases, input validation, debugging tools, breakpoint-based debugging, exception diagnosis, and program refinement. K5.6: AI-assisted and vibe-coding practices: requirement decomposition, prompt specification, code generation, code explanation, test generation, debugging support, validation, and responsible human review. K5.7: Desktop application mini-project architecture: user interface, program logic, external data access, testing, and refinement.	6 hrs
		A5.1: Develop Tkinter-based Login and Calculator applications using appropriate widgets, layout managers, event handlers, and input validation. A5.2: Construct a Wallpaper Viewer application that displays and changes images through a graphical interface. A5.3: Develop a News Application inspired by an Inshorts-style interface by integrating Tkinter GUI, classes, external APIs, and JSON data processing. A5.4: Apply introductory threading or multiprocessing concepts to maintain responsiveness during a time-consuming application operation. A5.5: Create test cases for GUI and data-processing applications; identify logical, runtime, and input-validation defects; debug and refine the program. A5.6: Use AI coding tools to generate an initial prototype from a clearly written requirement, then verify generated code through execution, test cases, debugging, and documented revisions. A5.7: Present a completed Python desktop mini-project demonstrating functional correctness, interface usability, external-data integration where applicable, and evidence of testing.	12 hrs
		Total	90 hrs

Unit / CO	CO1	CO2	CO3	CO4	CO5
Unit-I	3	1	0	0	0
Unit-II	2	3	0	0	0
Unit-III	1	2	3	0	1
Unit-IV	2	1	1	3	1
Unit-V	2	2	1	2	3

Examples of Practical Exercises

Practical Exercise 1: Arithmetic and Assignment Operator Calculator

Develop a Python program that accepts two numbers and performs addition, subtraction, multiplication, division, floor division, modulus, and exponentiation. Use assignment operators to update one of the values and display the revised result.
Deliverable: Commented Python script and sample input-output record.

Practical Exercise 2: Student Eligibility Checker

Develop a program that accepts attendance percentage, examination marks, and fee-status input. Use relational and logical operators with if-else statements to determine whether the student is eligible for an examination.
Include at least five different test cases.
Deliverable: Python script, test-case table, and output screenshots.

Practical Exercise 3: Number Classification and Multiplication Table

Write a Python program that:

- classifies a number as positive, negative, or zero;
- determines whether it is even or odd; and
- generates its multiplication table from 1 to 10 using a for loop.

Deliverable: Python script with sample outputs for at least three numbers.

Practical Exercise 4: Guessing Game Using Loops

Create a number-guessing game in which the user attempts to guess a predefined or randomly generated number. Use a while loop, conditional statements, a counter for limiting attempts, and break to end the game after a correct guess.
Use continue to handle an invalid guess where appropriate.
Deliverable: Working Python program, test record, and short note explaining the loop-control statements used.

Practical Exercise 5: String Formatting and Validation

Develop a program that accepts a name and email address. The program should:

- remove unwanted spaces;
- convert the name into title case;
- count the number of characters;
- extract initials from the name;
- identify the domain part of the email address; and

- validate whether the name contains only letters and spaces.

Deliverable: Commented Python script and sample input-output record.

Practical Exercise 6: List-Based To-Do Manager

Create a menu-driven to-do manager using a Python list. The program should allow the user to:

- add a task;
- display all tasks;
- update a selected task;
- delete a selected task; and
- exit the application.

Handle invalid task positions appropriately.
Deliverable: Python script, test cases, and a brief reflection on list operations used.

Practical Exercise 7: Dictionary-Based Student Record System

Develop a program that stores student records in a dictionary using enrolment number as the key. Each record should contain name, programme, semester, and marks.
Implement options to add, view, update, search, and delete student records. Test the program with at least three records.
Deliverable: Working Python program, sample output, and comparison note on the use of lists and dictionaries.

Practical Exercise 8: Functions and Recursion Practice

Write Python functions to:

- calculate the area of a circle, rectangle, and triangle;
- find the largest among three numbers;
- check whether a number is even or odd; and
- calculate factorial using recursion.

For the recursive factorial function, identify the base case and trace the function calls for one selected input.
Deliverable: Python script, recursion-trace table, and sample output.

Practical Exercise 9: Class-Based Library Book Manager

Create a Book class with attributes such as book title, author, accession number, availability status, and borrower name. Include methods to issue a book, return a book, and display book details.
Test the program with multiple book objects and include a condition that prevents issue of an already issued book.
Deliverable: Python script, object-output record, and class-design note.

Practical Exercise 10: File-Based Marks Analyzer with Exception Handling

Create a text file containing student marks, with one mark on each line. Develop a Python program that reads the file and calculates total, average, highest mark, and lowest mark.
Use try, except, else, and finally to handle missing files and invalid entries. Use the with statement for file operations. Include a suitable message when a non-numeric value is found.
Deliverable: Source text file, Python script, output summary, and exception-test record.

As this response is AI generated, , it may contain some inaccuracies or the information may sometime be obsolete or incomplete. You are advised to carefully verify the specific details from authentic sources.
At BSDU, the following problem-solving tasks and questions for **CS1105: Python Programming** are designed to move students beyond routine coding towards debugging, selecting suitable approaches, testing, and justifying decisions.

Example of Problem Solving Tasks

PST1. Incorrect Student Result Calculator

A student-result program gives incorrect percentages when marks are entered as decimal values.
Students will:

- trace the program to locate errors in formula, data type, or list processing;
- revise the calculation and grade-classification logic;
- test the program using at least five varied records; and
- submit the corrected program, test table, and a brief justification of corrections.

Expected output: Debugged result-processing program and validation report.

PST2. Infinite Loop in a Menu-Driven Application

A menu-driven Python program for a to-do list keeps repeating even after the user selects "Exit."
Students will:

- inspect the loop condition and menu-selection logic;
- decide whether to revise the loop condition, update the control variable, or use break;
- implement and test the corrected control flow; and
- document why the revised logic terminates reliably.

Expected output: Corrected menu-driven program, code trace, and test evidence.

PST3. Duplicate Inventory Records

A small inventory application stores item names in a list and begins showing duplicate entries, inconsistent quantities, and slow searching.
Students will:

- analyse the limitations of the existing list-based design;
- compare the use of a list, set, and dictionary for the stated requirements;
- redesign the data representation to prevent duplicate item identifiers;
- implement add, search, update, and display operations; and
- test the revised program with duplicate-entry attempts.

Expected output: Working inventory program, data-structure comparison, and test report.

PST4. Password Validation and Access Control

A login program permits weak passwords and crashes when users enter unexpected values.
Students will:

- define password-validation rules using string operations and logical operators;
- create a bounded-attempt login process using loops;
- use appropriate conditional logic for valid, invalid, and locked-account situations;

- test boundary cases such as blank input, short passwords, and repeated failures; and
- explain the role of break and continue in the solution.

Expected output: Validated login program, test-case table, and logic explanation.

PST5. Library Book Management Application

Develop and refine a console-based library book manager for storing book details and issue status. Students will:

- model books using a class with suitable attributes and methods;
- implement add-book, search-book, issue-book, return-book, and display-book operations;
- prevent issue of an unavailable book;
- use functions to organise repeated operations; and
- test the application with multiple book objects and exceptional user choices.

Expected output: Python application, class diagram or object design note, and demonstration record.

Examples of Problem-Solving Questions

PSQ1. Input Conversion Failure

Consider the code:
age = int(input("Enter age: "))
print("Eligible age:", age)
The program terminates when a user enters 18.5 or eighteen.

1. Analyse why the program fails for these inputs.
2. Propose an input-handling strategy suitable for age values.
3. Recommend suitable validation conditions for acceptable age input.
4. Justify whether integer or float conversion is appropriate for this application.

PSQ2. Selecting a Collection Type

A programme must store:

- a fixed set of course codes that should not change;
- a frequently updated list of tasks; and
- student records searchable by enrolment number.

1. Recommend an appropriate Python collection for each requirement.
2. Analyse the role of mutability in the choices.
3. Compare the suitability of lists, tuples, sets, and dictionaries for this scenario.
4. Justify how the proposed design supports efficient retrieval and updating.

PSQ3. Loop Design for Repeated Validation

A user must enter a number between 1 and 100. The program should continue asking until valid input is entered.

1. Analyse whether a for loop or a while loop is more suitable.
2. Design the logic for handling values outside the range.
3. Explain where continue or break may be appropriately used.
4. Evaluate the effect of failing to update a loop-control variable.

PSQ4. Recursive Factorial Failure

A student writes the following function:

```
def factorial(n):
    return n * factorial(n - 1)
```

1. Diagnose why this function does not terminate.
2. Propose an appropriate base case.
3. Trace the corrected function for factorial(4).
4. Evaluate how the function should respond to zero and negative inputs.

PSQ5. Debugging a String Palindrome Program

A program is intended to check whether a word is a palindrome, but it treats Madam and madam as different and gives incorrect results when spaces are present.

1. Analyse the causes of the incorrect output.
2. Propose suitable string-processing steps before comparison.
3. Develop the algorithm or structogram for the corrected solution.
4. Justify how the revised approach handles case differences and unwanted spaces.

As Confucius observed, "I do and I understand." These activities require learners to diagnose, decide, implement, test, and justify Python solutions—**Learn, Practice, Perform. Succeed.**

Course Name	Software Development with Python
Course Code	BTCS101

Study Material-I: Unit-Content-Wise Resources

B1: Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

B2: Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3rd ed.). No Starch Press.

B3: Ramalho, L. (2022). *Fluent Python* (2nd ed.). O'Reilly Media.

B4: Sedgewick, R., Wayne, K., & Dondero, R. (2015). *Introduction to programming in Python: An interdisciplinary approach*. Pearson.

B5: Lutz, M. (2010). *Programming Python* (4th ed.). O'Reilly Media.

Online Resources:

OR1: NPTEL course: https://nptel.ac.in/courses/106106145

Unit and Unit Name	Contents	Topic name	Primary Resource(s)	Additional Resources
Unit I: Python Environment, Data Representation, and Basic Program Development	K1.1, K1.2	Python interpreter, VS Code/Colab/Python Tutor, program structure, print(), comments, indentation, syntax/runtime errors	B1, Ch. 1–3	B2, Ch. 1, OR1
	K1.3	Variables, keywords, identifiers, literals, dynamic typing, dynamic binding	B1, Ch. 6, 11	B2, Ch. 2
	K1.4	Numeric, Boolean, string, and None data types; operators and expressions	B1, Ch. 5, 7	B4
	K1.5	Input processing, type conversion, formatted output,	B1, Ch. 10	B2, Ch. 2, 7

		simple computational logic		
	K1.6	AI-assisted code-generation workflow: prompting, review, execution, validation, correction	Not covered in the prescribed textbooks	OR1
Unit II: Decision Making, Iteration, and Logical Problem Solving	K2.1, K2.2	Arithmetic/relational/logical/assignment/membership/identity operators; Boolean expressions, precedence	B1, Ch. 5, 12	B2, Ch. 5
	K2.3	Conditional constructs: if, if-else, if-elif-else, nested conditions	B1, Ch. 12	B2, Ch. 5
	K2.4	Iterative constructs: while, for, nested loops, range(), sequence traversal	B1, Ch. 13	B2, Ch. 4, 7
	K2.5	Loop-control statements: break, continue, pass; sentinel/counter-controlled iteration	B1, Ch. 13	B4
	K2.6	Algorithm tracing, dry runs, test-case design, debugging logical errors	B1, Ch. 13–14	B4
Unit III: Strings, Collections, Modules, and Data Processing	K3.1	String creation, indexing, slicing, concatenation, formatting, comparison, string methods	B1, Ch. 7	B2; B3
	K3.2	Lists: creation, indexing, slicing, traversal, updating, searching, sorting, list methods	B1, Ch. 8	B2, Ch. 3–4; B3
	K3.3, K3.4	Tuples, sets, dictionaries; mutability, immutability, aliasing, shallow copying	B1, Ch. 8–9	B2, Ch. 6; B3
	K3.5	CRUD operations on lists and dictionaries for structured application data	B1, Ch. 8	B2, Ch. 6

	K3.6	Built-in functions and introductory modules: math, random, datetime	B1, Ch. 22	B4
	K3.7	Selecting suitable collection types for textual, tabular, unique, key-value data	B1, Ch. 9	B3
Unit IV: Functions, Functional Expressions, and Object-Oriented Programming	K4.1	User-defined functions, parameters, arguments, return values, local/global scope	B1, Ch. 16–18	B2, Ch. 8
	K4.2	Recursion, base conditions, recursive tracing, introductory memoization	B1, Ch. 19	B2, Ch. 8; B4
	K4.3	Lambda expressions; map(), filter(), reduce()	B1, Ch. 19	B2; B3
	K4.4	Classes, objects, attributes, methods, constructors, instance behaviour	B1, Ch. 27–28	B2, Ch. 9; B4
	K4.5	Encapsulation, inheritance, method overriding, polymorphism	B1, Ch. 29, 31	B2, Ch. 9; B3
	K4.6	Iterators and generators for controlled sequence processing	B1, Ch. 14, 20	B3
	K4.7	Modular program design using functions, classes, and imported modules	B1, Ch. 22–23	B4
	K5.1	Tkinter fundamentals: window, widgets, layout managers, event handling, labels, buttons, message boxes	B5, Ch. 7	—
	K5.2	GUI application design using classes and object-oriented event-driven programming	B5, Ch. 10	B2

Unit V: GUI Applications, APIs, Concurrency, Testing, Debugging, and AI-Assisted Development	K5.3	External APIs, HTTP requests, JSON data, integrating external data	B5, Ch. 13	B4
	K5.4	Threading and multiprocessing fundamentals; responsiveness, background tasks	B5, Ch. 5	—
	K5.5	Test cases, input validation, debugging tools, breakpoint debugging, exception diagnosis	B4	B1, Ch. 33–36
	K5.6	AI-assisted and vibe-coding practices: prompt specification, code generation/explanation, test generation	Not covered in the prescribed textbooks	OR1
	K5.7	Desktop application mini-project architecture: UI, program logic, external data access, testing	B5, Ch. 11	B4

Study Material-II: Unit-Content-Wise Notes

Study Material-III: Unit-Content-Wise Activities

UNIT-I

Activity 1: Learner Profile Input Program
Students create a Python program that accepts a learner's name, enrolment number, age, and percentage through input(). They store the inputs using meaningful identifiers and display a clearly labelled learner profile.
Students must add comments that explain:

- the purpose of the program;
- each input statement;
- type conversion used for age and percentage; and
- the output section.

Worksheet prompts

1. Why does input() return data in string form?
2. Which inputs require conversion before numerical use?
3. Identify two comments that explain *why* a code step is used rather than merely repeating the code.

Expected deliverable: A commented Python script named learner_profile.py and a screenshot or record of two test runs.

Activity 2: Input Validation Reflection Task
Students run the learner-profile program with suitable and unsuitable entries, such as a numeric age, a decimal percentage, and a non-numeric age. They observe the program's response when conversion is attempted and record the result.
They prepare a short reflection addressing:

- which input caused an issue;
- at which statement the issue occurred;
- why that input was unsuitable for the selected conversion; and
- how the program instructions could guide a user to enter appropriate data.

Expected deliverable: Test log with inputs, observed outputs or errors, and a brief reflection note.

Activity 3: Personalised Output and Variable Practice
Students write a program that creates variables for a learner's name, programme, semester, attendance status, and marks. They use print() to present the information in a readable format.
The program must demonstrate:

- output of text and variable values;
- use of multiple arguments in print();
- use of sep or end in at least one output statement; and
- meaningful variable names.

Students then change the values stored in variables and compare the revised output without altering the output structure.
Expected deliverable: Python script and two output records generated using different variable values.

Activity 4: Simple Billing Display Program
Students create a program that stores the name of an item, quantity, unit price, and total amount in variables. The program should display a simple bill using formatted and clearly labelled output. At this stage, values may be assigned directly in the program; calculation logic may be kept simple and limited to the learners' readiness.
Students review the code in pairs and identify:

- one variable name that could be improved;
- one output line that could be made more readable; and
- whether every displayed value has an appropriate label.

Expected deliverable: simple_bill.py, peer-review note, and revised version of the program.

Activity 5: Variable Rebinding Investigation
Students create a variable named value and assign an integer to it. They then reassign the same variable with a decimal number, a string, and a Boolean value. After every assignment, they display both the value and its type using type().
Students predict the output before running the program and then compare their prediction with the actual output.
Worksheet prompts

1. Which assignments changed the type associated with value?
2. Did Python require a separate declaration before each reassignment?
3. What does this observation indicate about dynamic typing and dynamic binding?

Expected deliverable: Annotated Python script, prediction sheet, and output record.

Activity 6: Debug and Explain Dynamic Behaviour
Students are given short programs containing variable reassignment. They trace each statement and identify the current value and type after every line. They then correct programs where an operation is attempted between incompatible types, such as combining a numeric value and text without appropriate conversion.
The activity should conclude with a brief pair discussion: dynamic typing can make initial program writing flexible, but why must programmers still inspect values and data types carefully?
Expected deliverable: Completed trace worksheet, corrected scripts, and a short explanation of the correction.

Activity 7: Mini Practice Task—Interactive Student Information Display
Students develop a concise interactive program that:

- accepts a learner's name, age, and percentage through input();

- converts age and percentage to suitable data types;
- stores each value in meaningful variables;
- uses comments to document the code;
- displays a labelled learner-information summary using print(); and
- demonstrates dynamic rebinding by assigning a revised value to one selected variable and displaying its updated type.

Students test the program using at least two sets of inputs, review it with a peer, and revise one aspect of documentation, naming, or output clarity.
Expected deliverable: Working Python script, test record, peer-review note, and a short reflective note on the difference between variable value and variable type.
As Alan Kay observed, "The best way to predict the future is to invent it." At BSDU, these activities support **Active and Experiential Learning** by taking students from input and output practice to careful observation of Python's dynamic behaviour. **Learn, Practice, Perform. Succeed.**

UNIT-II

Activity 1: Operator-Based Billing Calculator

Students develop a Python program that accepts item price, quantity, discount percentage, and GST percentage as inputs. The program calculates the subtotal, discount amount, taxable amount, GST amount, and final bill amount.
Students must use arithmetic and assignment operators appropriately. They should also add comments explaining the calculation logic and test the program with at least three different input combinations.
Deliverable: Commented Python script, three test-output records, and a brief calculation sheet.

Activity 2: Logical Operator and Eligibility Checker

Students create a program to check whether a student is eligible for a selected academic activity based on conditions such as minimum attendance, minimum marks, and fee-status confirmation.
The program should use relational and logical operators including and, or, and not. Students must prepare at least five test cases that produce different outcomes and explain why each case is accepted or rejected.
Deliverable: Python script, test-case table, and short explanation of the conditions used.

Activity 3: Indentation Error Diagnosis Task

Students receive Python code snippets containing incorrect indentation in if-else, nested if-else, and loop blocks. They identify the error, predict the likely outcome, correct the code, and execute the revised version.
Students then prepare one short incorrectly indented code example for peer review. The peer must identify and correct the error.
Deliverable: Error-analysis sheet containing faulty code, corrected code, explanation, and peer-review record.

Activity 4: Number Classification Program

Students write a program that accepts an integer and classifies it as positive, negative, or zero. The program should further determine whether the number is even or odd where applicable.
Students extend the program using nested conditions to classify marks into grade categories according to a faculty-provided grading rule.
Deliverable: Two Python scripts—number classifier and grade classifier—with sample input-output records.

Activity 5: Multiplication Table and Pattern Generator

Students use a for loop to generate the multiplication table of a user-entered number. They then create a simple numerical or star pattern using nested loops.
Students compare the use of a single loop and nested loops, recording the role of the outer and inner loop in the pattern-generation program.
Deliverable: Python scripts for multiplication table and pattern generation, with a short loop-trace note.

Activity 6: Repetition and Loop-Control Practice

Students develop three small programs:

- a program that repeatedly accepts numbers until the user enters 0;
- a program that skips negative values using continue;
- a program that ends a search when a specified value is found using break.

Students must also include one meaningful use of pass, with a comment describing why it is included.
Deliverable: Set of documented Python scripts and a comparison note on break, continue, and pass.

Activity 7: Guessing Game Application

Students develop a console-based Guessing Game. The program generates or stores a target number, accepts repeated guesses, and informs the user whether the guess is too high, too low, or correct.
The program must:

- use a while loop;
- use conditional statements;
- limit the number of attempts using a counter;
- use break when the correct answer is guessed; and
- display a final message showing success or unsuccessful completion.

Students test the application with different guessing sequences and refine the feedback messages for clarity.
Deliverable: Working Guessing Game script, test record, and short reflection on the control-flow logic used.

UNIT III

Activity 1: Student Name Formatting and Validation Task

Students accept a student name through input() and apply string operations to remove extra spaces, convert the name into title case, count characters, and display initials.
Students also check whether the entered name contains only alphabetic characters after spaces are removed. They document the string functions used and explain the purpose of each.
Deliverable: Python script, sample outputs, and a string-function usage note.

Activity 2: Text Slicing and Information Extraction Task

Students work with a sample registration code, email address, or sentence. They use indexing and slicing to extract specific elements such as programme code, year, domain name, first word, last word, or selected characters.
Students predict outputs before executing each slicing expression and compare the prediction with the actual result.
Deliverable: Python script, prediction-versus-output worksheet, and explanation of slicing expressions.

Activity 3: String Editing and Data Cleaning Challenge

Students receive a list of poorly formatted text entries containing unwanted spaces, mixed letter cases, repeated symbols, or inconsistent separators. They create a program to clean and standardise the entries using suitable string methods.
The final output should be formatted consistently and should preserve only meaningful information.
Deliverable: Data-cleaning script, before-and-after output record, and brief explanation of the selected string methods.

Activity 4: List-Based To-Do Manager

Students create a menu-driven to-do manager using a Python list. The application should allow a user to add tasks, view tasks, update a selected task, and delete a task.
Students must ensure that invalid list positions are handled appropriately. They should use loops to keep the program active until the user chooses to exit.
Deliverable: Working list-based to-do manager, test record, and code documentation.

Activity 5: Dictionary-Based Student Record Manager

Students develop a simple student-record application using dictionaries. Each record may include enrolment number, name, programme, semester, and percentage.
The application should support:

- adding a student record;
- viewing a record using enrolment number;
- updating a selected field; and
- deleting a record.

Students test the program with at least three records and reflect on why dictionaries are useful for structured information.
Deliverable: Python script, sample record output, and reflection note.

Activity 6: Mutability and Variable Referencing Investigation

Students create examples using lists, tuples, and dictionaries to observe the difference between mutable and immutable objects. They assign one variable to another, modify an object where possible, and observe whether changes are visible through both references.
Students prepare a trace note distinguishing copying from assigning a reference.
Deliverable: Annotated Python script, observation table, and short explanation of mutability and referencing.

Activity 7: Built-In Module Selection Task

Students are given short programming situations, such as generating a random quiz question, calculating the square root of a value, or working with a date. They identify an appropriate built-in module and develop a short program using the relevant imported feature.
Suggested contexts may include math, random, and datetime.
Deliverable: Three small Python scripts and a justification note for the module selected in each case.

UNIT IV

Activity 1: Function Design and Reuse Task

Students convert repeated program logic into reusable functions. They develop functions with parameters and return values for tasks such as calculating total marks, calculating simple interest, checking whether a number is even, and finding the larger of two values.
Students call each function with different inputs and compare the clarity of the function-based version with a single long script.
Deliverable: Python script containing multiple functions, test-output record, and brief reflection on code reuse.

Activity 2: Recursive Mathematical Functions

Students write recursive functions to calculate factorial and generate the Fibonacci sequence up to a selected limit. They trace the function calls for a small input and identify the base case and recursive case. Students then compare recursive and iterative approaches for one selected problem.
Deliverable: Python scripts, recursion-trace sheet, and comparison note.

Activity 3: Lambda, Map, Filter, and Reduce Practice

Students prepare a collection of numbers representing marks, expenses, or item prices. They use:

- a lambda expression to transform values;
- map() to apply an operation to all values;
- filter() to select values meeting a condition; and
- reduce() to calculate an aggregate result.

Students explain the role of each operation and verify the result through a manually calculated example.
Deliverable: Python script, input-output record, and concise explanation of each functional programming tool.

Activity 4: Class and Object Modelling Task

Students design a class to represent a simple real-world entity such as a Student, Book, Product, or BankAccount.
The class should include suitable attributes and at least two methods. Students create multiple objects, display their details, and invoke the methods to demonstrate object behaviour.
Deliverable: Python class-based program, object-output record, and class design note.

Activity 5: Library Book Issue Simulation

Students develop a small class-based application to manage a library book. The application should record book title, author, availability status, and borrower name where applicable.
Students implement methods to issue a book, return a book, and display its current status. They test the program with situations such as attempting to issue an already issued book.
Deliverable: Python script, test cases, and brief error-handling or decision-logic note.

Activity 6: Function Refactoring and Peer Review

Students receive a long Python program containing repeated code and unclear variable names. They reorganise the program into smaller functions with meaningful names, suitable parameters, and return values.
In pairs, students review each other's refactored program against the following criteria:

- logical separation of tasks;
- meaningful function names;
- appropriate use of parameters and return values;
- absence of unnecessary repetition; and
- readable documentation.

Deliverable: Original and refactored programs, peer-review checklist, and revision note.

Activity 7: Mini Application—Student Result Analyzer

Students create a menu-driven program using functions and, where appropriate, classes to store and analyse student marks.
The program should allow entry of marks, calculation of total and percentage, classification of result, and display of a formatted result summary. Students should divide the application into meaningful functions and explain the responsibility of each function or class method.
Deliverable: Working Python application, test-output record, program structure diagram, and reflection note.

UNIT V

Activity 1: Text File Creation and Reading Task

Students create a text file using the with statement and write basic student information, such as enrolment number, name, programme, and semester, into the file.
They then reopen the file, read its contents, and display the information on the screen. Students compare the effect of write mode and read mode.
Deliverable: Python script, created text file, and output record.

Activity 2: File Append and Update Practice

Students create a simple attendance or daily-expense file. They write initial entries into the file and then use append mode to add further records without deleting existing information.
Students inspect the file after every operation and explain the difference between overwriting and appending data.
Deliverable: Python script, text file containing multiple entries, and file-operation observation note.

Activity 3: Sales Data Processing from a File

Students prepare a text file containing numeric sales, marks, or expense values, with one value on each line. They write a program to read the values, calculate total, average, maximum, and minimum, and display a formatted summary.
Students should include at least one invalid or blank entry in the source file and observe how the program behaves before adding error-handling logic.
Deliverable: Source data file, Python script, output summary, and initial error observation.

Activity 4: Error-Tolerant File Processing Task

Students improve the file-processing program by using try, except, else, and finally blocks. The program should handle cases such as:

- file not found;
- invalid numeric data;
- division by zero where relevant; and
- unexpected user input.

Students document the purpose of each exception-handling block and test the program using at least three error situations.
Deliverable: Error-tolerant Python script, error-test record, and exception-handling explanation.

Activity 5: Custom Exception Design Task

Students define a simple custom exception for a meaningful condition, such as an invalid age, negative marks, invalid withdrawal amount, or insufficient attendance.
They create a program that raises the custom exception when the condition is violated and displays a clear user-friendly message.
Deliverable: Python script using a custom exception, test cases, and short explanation of when custom exceptions are useful.

Activity 6: Standard Library Module Practice

Students develop short programs using selected standard library modules:

- math for calculations such as square root, power, or ceiling value;
- random for generating quiz questions, random numbers, or a simple game element; and
- datetime for displaying the current date, calculating age from date of birth, or creating a dated record entry.

Students identify the import statement used and explain the role of the selected module.
Deliverable: Three module-based Python scripts and a module-usage note.

Activity 7: Mini Application—File-Based Contact Book

Students develop a console-based Contact Book application that stores names and phone numbers in a text file. The program should allow the user to add a contact, view all contacts, search for a contact, and append new entries without removing prior records.
The application must use:

- the with statement for file operations;
- functions to separate major tasks;
- exception handling for missing files or invalid input; and
- clear user messages and code comments.

Deliverable: Working Python application, text-file database, test record, and short reflection on the role of files in persistent data storage.